2st Lab with VTK (Visualization Tool Kit)

João Varela, 113780
Carolina Prata, 114246
Information Visualization, 2025
MSc in Computer Engineering, University of Aveiro

***Abstract*—This document provides a brief description of the work developed for the second lab of the Information Visualization course, with the objective of understanding how projections, lighting, and transformations work within VTK.**

I. Introduction

The Visualization Toolkit (VTK) is an open-source software framework extensively used in 3D computer graphics, image processing, and scientific visualization, offering robust tools for handling various data types, supporting visualization methods such as volume rendering, contouring, and streamline analysis, while also enabling the use of multiple actors, renderers, shading options, textures, transformations for object orientation and positioning, and the implementation of observers and callbacks.

II. Development

The laboratory assignment provided by the course instructor comprises several exercises covering the following topics:

- Multiple actors;
- Multiple renderers;
- Shading options;
- Textures;
- Transformations;
- Observers and callbacks;

A. Multiple actors

For the first exercise, it was required to add multiple actors to one render in `cone.py`. In this case, we were asked to create two identical cones with different colors. We used two approaches to manipulate the properties of each cone: in the first case, we set each property individually, while in the second case, we created a `vtkProperty` to store and modify the properties and then assigned it to the actor. Additionally, we were asked to set the opacity of both cones to 0.5. We changed the background to white `SetBackground(1.0, 1.0, 1)` to better visualize the different opacities.

The following code demonstrates how we set the properties of the first cone individually:

```
coneActor.GetProperty().SetColor(0.2, 0.63, 0.79)
coneActor.GetProperty().SetDiffuse(0.7)
coneActor.GetProperty().SetSpecular(0.4)
coneActor.GetProperty().SetSpecularPower(20)
coneActor.GetProperty().SetOpacity(0.5)
```

For the second cone, we used a `vtkProperty` to store and modify the properties, which were then applied to the actor. We also set its position to `(0, 2, 0)`:

```
property = vtkProperty()
property.SetColor(1.0, 0.3882, 0.2784)
property.SetDiffuse(0.7)
property.SetSpecular(0.4)
property.SetSpecularPower(20)
property.SetOpacity(0.5)

coneActor2 = vtkActor()
coneActor2.SetMapper(coneMapper)
coneActor2.SetPosition(0, 2, 0)
coneActor2.SetProperty(property)
```

Both cones were given an opacity of 0.5, resulting in a semi-transparent effect. The second cone was also translated by 2 units along the Y-axis, as seen in the `SetPosition` method.

Fig. 1. Comparison of the cone's appearance with reduced opacity (0.5) and full opacity (1).

We concluded that by reducing the opacity, the other faces of the cones became visible, giving the appearance of acrylic. Apart from this, the application of properties produced the same result regardless of the method used to modify them for each cone.

B. Multiple renderers

This exercise involved displaying two renderers within a single window, done in `renderers.py`. To achieve this, we used the cone example from the previous lab and divided the window (set to 600 x 300 pixels) into two sections using the SetViewport method. The second renderer's camera was rotated 90 degrees in azimuth. Additionally, we modified the background colors for each renderer to better distinguish them.

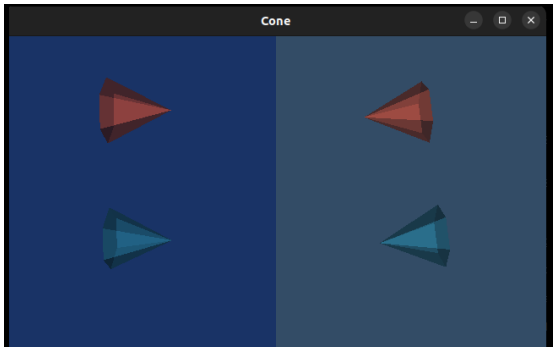

Fig. 2. Result of the execution of the double renderer.

C. Shading options

For this exercise, done in `shadings.py`, the cones were replaced by spheres in order to explore and apply different shading techniques available in VTK. To make the differences between shading models more noticeable, the sphere resolution was reduced by setting both the `PhiResolution` and `ThetaResolution` to 10. Instead of using two renderers and two spheres as initially required, we implemented four renderers, each displaying a single sphere.

Each renderer was configured with a different shading model: one sphere used the default shading, while the remaining spheres were rendered using `SetInterpolationToFlat`, `SetInterpolationToGouraud`, and `SetInterpolationToPhong`, respectively.

This setup allowed for a direct visual comparison of the effects produced by each shading method.

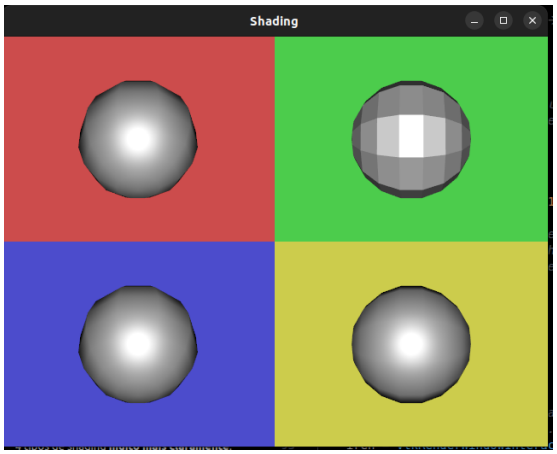

Fig. 3. Result of the execution of the shading exercise.

We observed that VTK applies Gouraud shading by default. The results showed that Gouraud and Phong shading often produce similar visual outcomes; however, Phong shading generally provides smoother lighting transitions and more realistic specular highlights. In contrast, flat shading is the most visually distinct, as it applies uniform lighting to each polygon, resulting in a clearly faceted appearance.

D. Textures

In this exercise, a plane was created using `vtkPlaneSource` and textured with an image in `textures.py`. The plane was defined as a 10×10 square by specifying an origin at $(0, 0, 0)$ and two edge points at $(10, 0, 0)$ and $(0, 10, 0)$. The Lena image was loaded using `vtkJPEGReader` and applied to the plane through a `vtkTexture` object. The texture was then assigned to the plane's actor using `SetTexture()`, mapping the image onto the surface and allowing it to be rendered correctly within the scene.

The result was a plane with the image as the texture as follows:

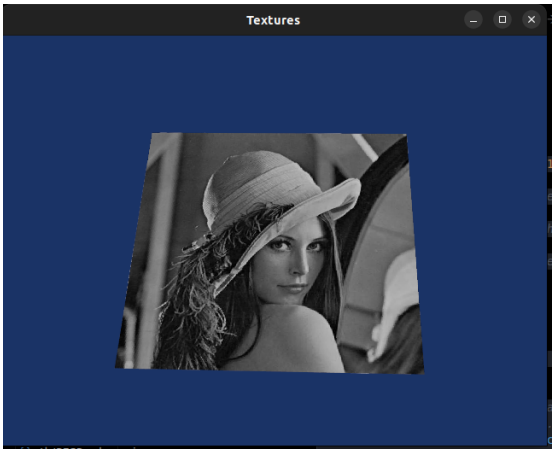

Fig. 4. Result of the execution of the transformation exercise.

When the dimensions of the plane are modified using the `SetOrigin`, `SetPoint1`, and `SetPoint2` methods, the applied texture automatically scales to cover the entire surface of the plane. As a result, the texture stretches or compresses to fit the new geometry, regardless of the plane's size or aspect ratio

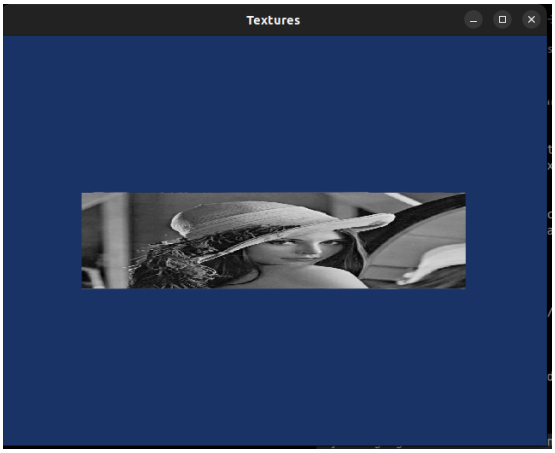

Fig. 5. Result of the execution of the transformation exercise.

E. Transformations

This exercise demonstrates the construction of a textured cube in VTK using individual planar faces in `transformations.py`. Each face of the cube is generated through a dedicated function, `render_cube_face`, which creates a plane and applies the necessary geometric transformations to position it correctly in 3D space. The transformations are performed using `vtkTransform`, combining rotations and translations to orient and place each face. Rotations are defined using an angle–axis representation, where the rotation angle is specified in degrees and the axis is defined by a three–component vector.

The transformed geometry is processed using `vtkTransformPolyDataFilter`, which applies the transformation to the polygonal data. For texture mapping, `vtkJPEGReader` is used to load an image for each face, and `vtkTexture` applies the corresponding texture to the plane. By creating six planes with distinct transformations and textures, a complete cube is assembled, with each face displaying a different image.

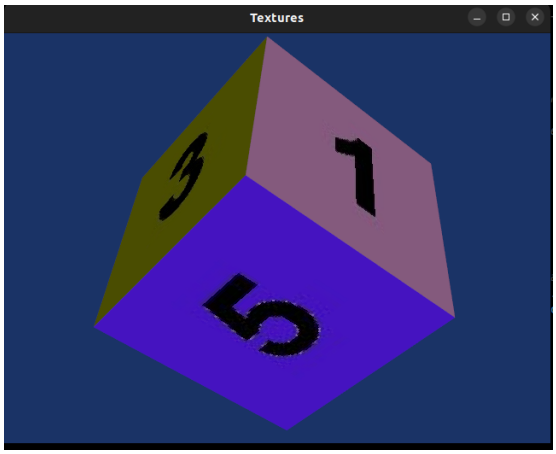

Fig. 6. Result of the execution transformations.py file

F. Observers and callbacks

In the final part of the lab, we explored VTK's mechanism for defining callback functions and associating them with objects through observers. To achieve this, a new file named `callbacks.py` was created based on the cone example from the first lab. A custom callback class, `vtkMyCallback`, was implemented to react whenever a specified event occurs during the rendering process.

When an event is triggered, the callback prints diagnostic information to the terminal, including the class name of the object that generated the event, the event type, and the current camera position in 3D space. By registering the callback with `vtkCommand.AnyEvent`, it was possible to capture and log multiple events occurring throughout the rendering cycle, such as `StartEvent`, `EndEvent`, `ModifiedEvent`, and `ResetCameraClippingRangeEvent`.

The observed output shows that `StartEvent` is triggered at the beginning of each rendering cycle, while `EndEvent` occurs once rendering is complete. By changing the observer to listen only to specific events (e.g., `vtkCommand.StartEvent`, `vtkCommand.EndEvent`, or `vtkCommand.ResetCameraEvent`), the callback mechanism can be tailored to respond selectively, enabling fine-grained control over event handling and effective debugging within the VTK rendering pipeline.